

TriAxialKV: Toward Extreme Low-Precision KV-Cache Quantization for Agentic Inference Tasks

Hanzhang Shen¹ Haoran Wu¹ Yiren Zhao² Robert Mullins¹
¹ University of Cambridge ² Imperial College London

Abstract

Agentic workloads have emerged as a major workload for LLM inference. They differ significantly from chat-only workloads, requiring long-context processing, the ability to handle multimodal inputs, and structured multi-turn interactions with tool calling capabilities. As a result, their context exhibits structure that can carry different importance along three key axes: temporal recency to the current turn, modality such as text or image tokens, and semantic role such as user queries, tool calls, observations, or reasoning. These axes capture distinct token behaviors and lead to different sensitivities to KV-cache compression. However, existing KV-cache quantization methods are typically homogeneous or exploit only heterogeneity on a single dimension, such as temporal proximity or modality, overlooking the interactions among them. To this end, we introduce **TriAxialKV**, a novel mixed-precision KV-cache quantization scheme that assigns each token a triaxial tag, calibrates per-tag sensitivity, and allocates INT2/INT4 bitwidths under a fixed memory budget. We implement TriAxialKV as an end-to-end serving system, comprising calibration, mixed-precision quantization and memory management, and custom fused Triton decode kernels. When using Qwen3-VL-32B-Thinking as a computer-use agent operating the OSWorld, TriAxialKV matches the accuracy of SGLang with BF16 KV cache while supporting $4.5\times$ KV cache size and achieving 30% higher end-to-end throughput, when running on real GPU systems.

1 Introduction

Large language models (LLMs) have rapidly moved beyond single-turn chatbots [46, 6] to *agentic* workloads such as web search [9, 17], computer use [2, 1], and code generation [10, 27, 32], where the model autonomously plans, invokes tools, consumes multi-modal inputs, and reasons over the resulting feedback across many turns. These workloads require the model to attend over very long, multi-turn contexts, causing the key-value (KV) cache to grow rapidly and become a dominant memory bottleneck in LLM inference. As the KV cache expands, it can quickly saturate GPU memory, thereby limiting the achievable batch size and throughput [39, 13]. For instance, when running a LLaMA-3-70B model on the OSWorld benchmark [41], the KV cache can reach approximately 100K tokens. At FP16 precision, the KV cache alone requires around 30 GB of memory for a single batch, consuming a substantial fraction of the HBM capacity of current GPUs.

KV-cache compression has therefore become essential for efficient agentic serving. A wide range of methods have been proposed, including uniform low-bit quantization [20, 4], asymmetric quantization [26, 34], token-eviction strategies that exploit attention sparsity [47, 7, 48], and adaptive bitwidth allocation along the layer depth or generation axis [19]. However, these methods largely treat the KV cache as a homogeneous tensor and apply a single compression method or policy across tokens, ignoring the fact that individual tokens differ substantially in how sensitive they are to quantization error. This assumption is particularly ill-suited to agentic workloads, where the prefill is highly heterogeneous: tokens can originate from 1) different *modalities* (e.g., text, image), 2) arrive at

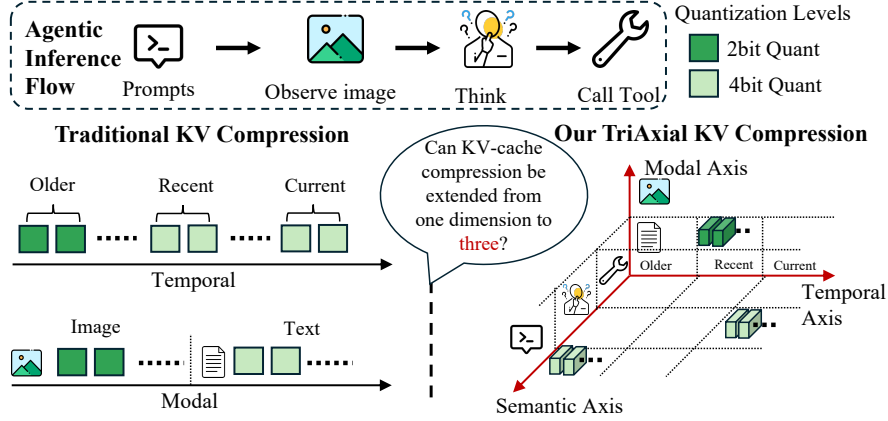


Figure 1: Comparison of single-axis KV-cache compression methods, including PM-KVQ [25] for temporal compression and VL-Cache [36] for modality-aware compression, with TriAxialKV, which holistically compresses the KV cache by jointly considering temporal, modal, and semantic axes.

different *times* in the conversation (system prompts, early user turns, recent tool outputs), and 3) play different *semantic* roles (instructions, reasoning, tool-call arguments, tool responses).

Recent works have begun to exploit individual dimensions for KV-cache compression. For instance, ThinKV considers the semantic role of tokens [31], VL-Cache and LOOK-M classify tokens based on modality [36, 37], and PM-KVQ categorizes reasoning tokens according to temporal recency [25].

However, they *treat these dimensions in isolation rather than jointly modeling the temporal, modal, and semantic structure of agentic traces*. As a result, tokens with substantially different compression sensitivities may be assigned the same bitwidth, leading to accuracy degradation under aggressive KV-cache compression. To address this gap, we propose **TriAxialKV**, a KV-cache quantization framework that jointly reasons about the *temporal*, *modal*, and *semantic* properties of each token to decide its individual bitwidth, enabling more aggressive compression at the same accuracy.

In summary, the main contributions of this work are:

- Through systematic profiling of agentic prefills across function-calling and computer-use workloads, we find that per-token KV quantization sensitivity spans more than an order of magnitude, and that this variation is captured almost entirely by three orthogonal axes: *temporal recency*, *modality*, and *semantic role*. To our knowledge, this is the *first* study to identify modality and temporal recency, alongside semantic role, as structural axes of KV quantization.
- Building on this discovery, we develop a *taxonomy-driven calibration and allocation framework*: a chat-template-based tagger that labels every prefill token in a single pass without any model inference, a calibration procedure that measures per-tag attention-output distortion from real prefill captures, and a per-tag bitwidth allocator that solves the INT2/INT4 assignment problem.
- We deliver an *end-to-end serving system* that integrates the taxonomy, allocator, and mixed-precision memory pool with fused Triton flash-decoding kernels into SGLang [49]. On Qwen3-VL-32B [5] running OSWorld [41], our system fits **4.5× the KV cache** and serves **1.3× the end-to-end throughput** of the BF16 baseline while matching its accuracy. On Qwen3-32B [42] running BFCL Memory [29], it stays within 0.7 points of BF16, where uniform-precision baselines such as KIVI [26] and SGLang FP4 [49] drop by 4–5 points.

2 Related Work

KV cache compression To allow for larger batch sizes and longer context windows, a wide range of work explored KV cache compression. KIVI [26], a seminal work on KV cache quantization, performed two-bit per-channel quantization for keys and per-token quantization for values to account for their distribution characteristics. KVQuant [19] targets sub-4-bit precision through pre-RoPE key quantization, non-uniform datatypes, and dense-and-sparse outlier isolation. ZipCache [18]

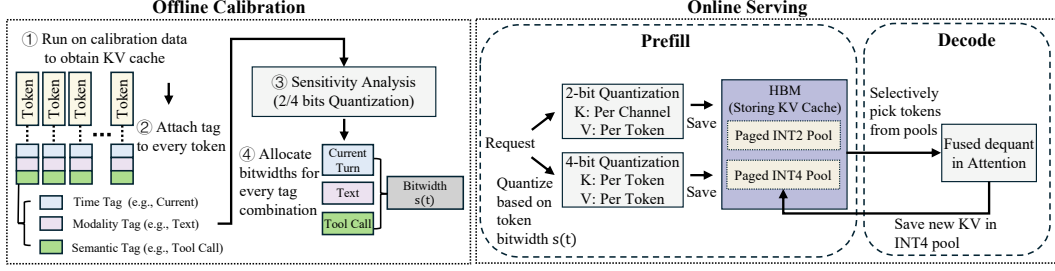


Figure 2: Overview of the TriAxialKV compression flow. During the prefill stage, KV entries are quantized and stored in either the INT2 or INT4 pool based on the per-token bitwidth assigned from the token’s triaxial tags. The selective KV selection process is detailed in Section 3.2.3, while the KV quantization scheme is described in Section 3.1.

identifies salient tokens via channel-separable scoring to enable aggressive compression of less-important tokens. QServe [24] co-designs W4A8KV4 quantization with a serving system to make the theoretical memory savings translate to measured throughput on commodity GPUs. A recent line of work introduces mixed-precision allocation along a single structural axis. PM-KVQ [25] designed a mixed precision quantization scheme by gradually lowering the bit-width of KV cache of each block. ThinKV [31] introduces a binary thought / non-thought split for reasoning models. ChanMix [23] allocates bits along the channel axis, and SmallKV [48] uses an auxiliary small model to compensate for compression error. A complementary line of work explored token eviction for KV cache compression [47, 40, 16, 22, 8, 15].

Efficient LLM serving systems A substantial body of work has addressed system-level challenges in LLM serving. Orca [44] introduced iteration-level scheduling to allow requests to join or leave a batch at each token generation step, maximizing GPU utilization. Sarathi-Serve [3] extended this with chunked prefills and stall-free scheduling to balance throughput and tail latency. FlashAttention [12, 11, 33, 45] utilized tiling to reduce the memory reads and writes between the GPU high bandwidth memory (HBM) and on-chip SRAM, facilitating efficient attention computation. vLLM [21] introduced PagedAttention, applying virtual memory paging to manage KV cache and reduce fragmentation. SGLang [49] designed a radix tree structure to organize KV cache and enabled prefix sharing among requests. A complementary line of work disaggregates the prefill and decode phases onto separate GPU pools, including DistServe [50], Splitwise [28], and Mooncake [30].

3 Method

An agentic prefill request consists of different segments, such as the system prompt, user queries, tool calls, observations, etc. We find that these segments exhibit structured variation in their sensitivity to KV cache quantization along three axes: *temporal recency*, *modality*, and *semantic role*. This variation enables a **sensitivity-aware allocation** that assigns bits to segments defined by the three axes above, subject to a bitwidth budget. Specifically, our system has two stages. First, an offline calibration stage measures the change in attention output induced by quantizing each segment to two and four bits, producing a sensitivity table over the tag space. Second, the serving system tags the prefill tokens and follows this table to allocate bitwidths to segments. We maintain two paged memory pools for INT2 and INT4 quantization, and implemented a fused Triton decode kernel that runs flash decoding with on-the-fly dequantization of both bitwidths.

3.1 Tri-Axial Quantization

We divide the tokens of an agentic prefill into segments along three orthogonal axes, and assign each token a tag from the joint tag space

$$S = \mathcal{A}_{\text{temporal}} \times \mathcal{A}_{\text{modal}} \times \mathcal{A}_{\text{semantic}},$$

where each axis describes a distinct property of the segment.

Table 1: Semantic axis values in our tag taxonomy.

Tag	Description
<code>inst</code>	system prompts and tool schemas
<code>user</code>	user-prose tokens
<code>assistant</code>	assistant-prose tokens outside any reasoning or tool bracket
<code>reasoning</code>	chain-of-thought spans inside <code><think></code> brackets
<code>tool_call</code>	tool invocations inside <code><tool_call></code> brackets
<code>obs</code>	environment feedback (tool outputs, screenshots)
<code>delim</code>	chat-template scaffolding

Temporal. $\mathcal{A}_{\text{temporal}} = \{\text{older}, \text{turn_m2}, \text{turn_m1}, \text{current}\}$. We divide tokens by their distance from the most recent turn, where a turn runs from one user message up to (but not including) the next. `current` covers tokens in the most recent turn; `turn_m1` and `turn_m2` cover the previous two turns; `older` includes everything earlier. This axis captures the decay of attention over turns: intuitively, older turns would contribute less to current attention, so they could in principle tolerate more aggressive quantization compared to more recent turns.

Modal. $\mathcal{A}_{\text{modal}} = \{\text{text}, \text{image}\}$. Tokens produced by the visual encoder and by the language-model embedding table have different statistics, such as per-channel variance and sparsity. Therefore, they could respond differently to low-bit quantization.

Semantic. $\mathcal{A}_{\text{semantic}} = \{\text{inst}, \text{user}, \text{assistant}, \text{reasoning}, \text{tool_call}, \text{obs}, \text{delim}\}$. These are functional roles in the agent loop, summarized in Table 1.

Combinations that do not occur in practice, such as `image|reasoning`, simply have zero token count and drop out of the optimization. All three axes are detectable from the chat template, with no content understanding required, which allows the tagger to run fast in the request preprocessor.

3.1.1 Allocation Setup

Consider a request whose prefill contains N tokens. Each token t contributes a key and value vector $k_t, v_t \in \mathbb{R}^d$ to the KV cache, and will be assigned a tag $s(t) \in \mathcal{S}$. We assign each tag k a bitwidth $b_k \in \{2, 4\}$ that applies uniformly to all tokens sharing that tag. Our goal is to minimize the perturbation of the attention output subject to a target average bitwidth across the prefill tokens.

3.1.2 Allocation Objective

Let o_i denote the full-precision attention output at position i , and $\tilde{o}_i(\mathbf{b})$ the output when KV vectors are quantized according to a per-tag bitwidth allocation $\mathbf{b} = (b_k)_{k \in \mathcal{S}}$. We aim to find the allocation $\mathbf{b}$ that minimizes the expected mean-squared error (MSE) of attention output:

$$\mathcal{L}(\mathbf{b}) \triangleq \mathbb{E}_i [\|o_i - \tilde{o}_i(\mathbf{b})\|_2^2]. \quad (1)$$

We use attention output MSE rather than raw KV quantization MSE because the two are not monotonically related: a token with small KV quantization error can still contribute large output error if attention concentrates on it, and conversely large KV error on a rarely-attended token has negligible effect. The nonlinear softmax and value mixing make output MSE the right quantity to minimize.

To make this MSE tractable, we expand the attention output to the first order in the quantization noise δ , yielding an expression linear in the per-token noise. We assume δ is zero-mean and uncorrelated across positions. Taking the first-order Taylor expansion of the attention output, we have

$$\hat{\mathcal{L}}(\mathbf{b}) = \sum_{k \in \mathcal{S}} D_k(b_k), \quad (2)$$

where $D_k(b)$ is the output MSE when only tokens with tag k are quantized to b bits and all other tokens remain full precision. We measure $D_k(b)$ directly from calibration data: on a calibration capture of KV cache, quantize only tokens with tag k at bitwidth b , replay attention, and record the output MSE against the full-precision reference. $D_k(b)$ depends only on the tag k and its assigned

bitwidth b . Therefore, our analysis produces a table of $2 \cdot |\mathcal{S}|$ distortion values for each tag for each bitwidth, which is reused for different average bitwidth budgets during the optimization later.

3.1.3 Optimization

Let $N_k = |\{t : s(t) = k\}|$ denote the token count for tag k in the KV cache. To estimate N_k , we have two cases. For `inst`-semantic tag that includes the system prompt and tool schemas, tokens are shared across most, if not all, requests under prefix caching, and the cache stores a single copy. Therefore, we take N_k as the median of token count in `inst`-semantic segments across calibration requests. For all other tags, we sum the prefill token count across all calibration requests. With target average bitwidth $B \in [2, 4]$, the allocation problem is:

$$\min_{\mathbf{b} \in \{2,4\}^{|\mathcal{S}|}} \sum_{k \in \mathcal{S}} D_k(b_k) \quad \text{s.t.} \quad \sum_{k \in \mathcal{S}} N_k b_k \leq B \sum_{k \in \mathcal{S}} N_k. \quad (3)$$

Upgrading tag k from 2 to 4 bits costs $2N_k$ extra bits in the cache and reduces the total MSE by $D_k(2) - D_k(4)$. We can define the per-bit gain of this upgrade:

$$\rho_k \triangleq \frac{D_k(2) - D_k(4)}{2N_k}. \quad (4)$$

When $|\mathcal{S}| \leq 22$, it is tractable to enumerate the full $2^{|\mathcal{S}|}$ allocation configurations and select the one with the minimal total MSE. For larger tag spaces, we take a greedy approach: sort tags by ρ_k in descending order, walk the sorted list, and upgrade each tag whose extra cost $2N_k$ fits in the remaining budget. Algorithm 1 in appendix A describes our greedy allocation.

3.1.4 Calibration

For each target workload, we take 5% of the dataset to form a calibration set, and use it to compute the per-tag bitwidth allocation in three stages.

KV capture. We run the calibration set through the serving system to capture the KV traces. For each model, we select four to six layers, evenly spread across the model’s depth, and register forward hooks on each layer’s QKV projection. The hooks fire on prefill only, capturing Q for the new tokens and KV of the prefill sequence.

Sensitivity analysis. For each capture and each tag k active in that capture, we quantize the KV of tokens tagged with k using bitwidth $b \in \{2, 4\}$, compute the attention output, and record the MSE against the full-precision reference to get $D_k(b)$. This yields a raw $D_k^{\ell,r,h}(b)$ for every layer ℓ , request r , and head h . We aggregate these into a final sensitivity score $D_k(b)$ using the following logic:

1. **Max across heads.** We take the maximum error across heads to ensure that no single attention subspace becomes a bottleneck, keeping the most sensitive head functional.
2. **Mean across requests.** We average the results over all calibration requests to ensure the metric is representative and stable across different inputs.
3. **Sum across layers.** We sum the errors across all layers to capture their cumulative impact, assuming that quantization noise is additive.

Bitwidth allocation and budget selection. The aggregated sensitivities $D_k(b)$ and token counts N_k are passed to the optimization algorithm (Section 3.1.3) to produce the per-tag allocation $\mathbf{b}^*$ for a target average bitwidth B . We determine the budget B by sweeping $B \in [2, 4]$ on the calibration set. We select the smallest B where performance remains stable and comparable to the baseline. Since both $D_k(b)$ and N_k are pre-computed, re-running bitwidth allocation is highly efficient.

3.2 System Design and Implementation

This section describes the serving system that realizes the bitwidth allocation from Section 3.1. We cover the quantization scheme and buffer layout, the mixed-precision memory pool, per-request lifecycle, and the fused decode kernel that unpacks and dequantizes KV on the fly.

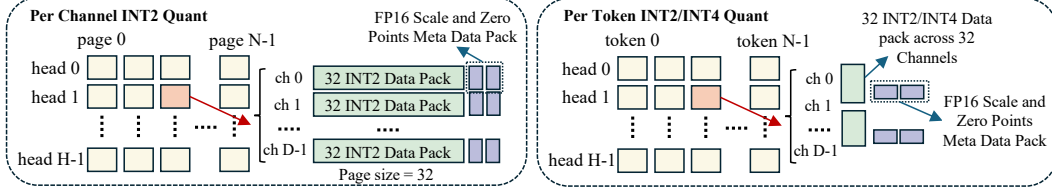


Figure 3: KV cache buffer layouts. Left: per-channel INT2 keys, indexed by (page, head). Right: per-token INT2 values and INT4 KV, indexed by (token, head). Page size matches the quantization group size $G = 32$. All buffers have the UINT8 data type.

3.2.1 Quantization Scheme and Buffer Layout

Bitwidths and quantization axes. Figure 3 shows our quantization scheme and packed buffer layout. We use asymmetric groupwise quantization scheme with group size $G = 32$ throughout. For INT4, we quantize **both keys and values per-token**. Channels in a head are split into groups of G elements, each quantized independently with its own (scale, zero-point). For INT2, we **quantize keys per-channel** and **values per-token**. Keys in a quantization group share one (scale, zero-point) pair *for each channel*. We quantize INT2 keys per-channel because, at low bitwidth, high-magnitude outliers in some channels inflate the scale and destroy resolution across other channels under per-token quantization [26]. Values are well-behaved under per-token quantization at both bitwidths.

Buffer layout. For per-channel quantization used by INT2 keys, the buffer is indexed by (page, head). As shown in Figure 3, each entry holds the quantized keys for all G tokens of the page under that head, grouped by channel, along with scales and zero-points. This ensures that the keys in a channel for G tokens sit in contiguous memory for coalesced loads. For per-token INT4 KV and INT2 values, each buffer is instead indexed by (token, head). Each entry stores the quantized data for all channels under one (token, head), followed by their scales and zero-points. All buffers are UINT8 arrays, with INT2 and INT4 data packed below the byte boundary.

Residual policy for INT2 keys. Per-channel quantization of INT2 keys requires the scale and zero-point of a channel to be shared across a group of G tokens. For the residual tokens that do not form a group, we *route them through the INT4 per-token path* instead. In KIVI [26], the residual tokens are stored in full-precision in a dedicated buffer, but we did not take this approach for two reasons. First, adding a residual buffer complicates memory allocation and further complicates the design of the attention kernel over three precisions. Second, the buffer holds 128 tokens’ KV in full precision in every layer. This adds a considerable overhead and erodes the memory savings that our INT2 path is meant to deliver, especially when there are many short requests. Keeping the residual tokens at the INT4 path makes memory management simple and reduces overhead.

3.2.2 Mixed-Precision Memory Pool

Shared address space. Both the INT2 and INT4 memory pools *share a single virtual address space*, split by an offset. The offset is set at startup from the calibrated average bitwidth B , which gives us the ratio of INT2 and INT4 tokens. Addresses below the offset index the INT2 pool, and addresses at or above the offset index the INT4 pool. Downstream components, such as the attention backend, can infer a token’s precision with a single comparison against the offset.

Memory allocation. We use one page allocator for each of the pools to handle its own memory allocation and freeing. When a request enters the scheduler, the bitwidth tagger produces a per-token bitwidth array. INT2 and INT4 tokens are routed to their respective allocators for memory allocation, and each allocator returns free page indices from its own list. No cross-pool coordination is required.

3.2.3 Request Lifecycle

Tagging. At serve time, each incoming request is tagged on the CPU during request scheduling and *assigned an array of per-token bitwidths* that propagates to the downstream memory allocator and attention backend. Specifically, we run the triaxial tagger from Section 3.1 on the tokenized

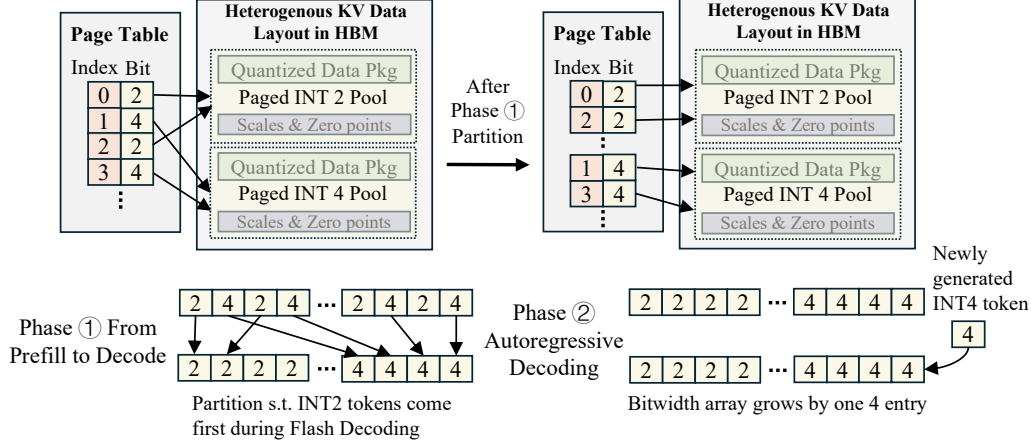


Figure 4: Per-request page table management. Phase 1 partitions page table entries so INT2 pointers precede INT4, enabling bitwidth-homogeneous flash-decoding splits; Phase 2 generates INT4 tokens autoregressively during decode and grows the page table by one entry per step.

input to produce a per-token tag code. Each tag code is looked up in the bitwidth allocation map $\mathbf{b}^* : \mathcal{S} \rightarrow \{2, 4\}$ from calibration, yielding a per-token bitwidth array attached to the request. To keep the tagger general across models and workloads, tagging is driven entirely by chat-template structure. We use special-token IDs from the tokenizer, such as role markers, message delimiters, and structural brackets for thinking and tool calls, to divide the sentence with a single scan.

Prefill. For prefill, we implement a Triton [35] gather-dequant kernel that reads the matched-prefix slot indices from the dual memory pool, unpacks each slot’s payload, dequantizes it into FP16, and writes the results into a buffer. The buffer is then passed to FlashInfer [43] FP16 prefill attention kernel without further modification. Queries, keys, and values for the newly-computed tokens are passed directly to the prefill kernel alongside the dequantized prefix.

Decode. Once prefill completes, the request transitions to decode. As shown in Figure 4, each request’s KV state is tracked in a per-request page table that records per-token pointers to the memory pool. During phase 1, we *partition* the pointers such that all INT2 tokens’ pointers come first before proceeding with decoding. This is because our fused decode kernel follows the flash-decoding design, where the KV is processed as parallel contiguous splits, merged via online log-sum-exp. Our partition ensures that the leading splits handle the INT2 KV and the trailing ones handle INT4. Each split is bitwidth-homogeneous and runs a single dequantization path. In phase 2, each autoregressive decode step generates a new INT4 token. Our fused dequantization-attention kernel has two paths: INT2 and INT4. Each path loads only the required KV data, unpacks them via bit shifts, and applies dequantization on the fly during attention computation. Keys and values of the generated token are written to the INT4 pool, and its address is appended to the page table.

4 Experimental Results

4.1 Experimental Setup

Datasets and Models. We evaluate our method on Berkeley Function Calling Leaderboard (BFCL) [29] and OSWorld [41] because they are two dominant agentic regimes: text-only function calling with structured tool schemas in BFCL Memory, and multimodal computer-use trajectories with screenshot observations in OSWorld. On BFCL, we evaluate on Qwen3 [42] (14B, 32B, and 235B-A22B-Instruct) and Falcon3-10B-Instruct [14]. On OSWorld, we evaluate on Qwen3-VL [5] (8B-Thinking, 32B-Thinking) and InternVL3.5-38B [38].¹

¹Licenses for all models, benchmarks, and libraries used in this work are listed in Appendix E.

Table 2: Task accuracy (%) on BFCL Memory across four models. Gray numbers in brackets indicate deltas relative to the BF16 baseline. TriAxialKV Mixed stays within 1.1 points across all models.

Method	Falcon3-10B-Instruct	Qwen3-14B	Qwen3-32B	Qwen3-235B-A22B-Instruct-2507
SGLang BF16 [49]	24.00 (+0.00)	25.11 (+0.00)	25.78 (+0.00)	23.78 (+0.00)
SGLang FP4 [49]	28.22 (+4.22)	18.00 (-7.11)	20.22 (-5.56)	23.56 (-0.22)
KIVI [26]	22.89 (-1.11)	20.67 (-4.44)	21.56 (-4.22)	23.33 (-0.45)
TriAxialKV INT4 (Ours)	22.00 (-2.00)	24.89 (-0.22)	24.67 (-1.11)	23.11 (-0.67)
TriAxialKV Mixed (Ours)	24.00 (+0.00)	24.22 (-0.89)	25.11 (-0.67)	22.67 (-1.11)

Table 3: Task accuracy (%) on OSWorld across multimodal models. Gray numbers in brackets indicate deltas relative to the BF16 baseline. TriAxialKV Mixed maintains the accuracy of the BF16 reference, with a substantially lower memory budget.

Method	Qwen3-VL-8B-Thinking	Qwen3-VL-32B-Thinking	InternVL3.5-38B
SGLang BF16 [49]	34.02 (+0.00)	39.20 (+0.00)	12.22 (+0.00)
SGLang FP4 [49]	29.52 (-4.50)	40.54 (+1.34)	12.13 (-0.09)
KIVI [26]	34.61 (+0.59)	41.49 (+2.29)	12.26 (+0.04)
TriAxialKV INT4 (Ours)	33.92 (-0.10)	41.43 (+2.23)	13.11 (+0.89)
TriAxialKV Mixed (Ours)	35.78 (+1.76)	40.59 (+1.39)	12.77 (+0.55)

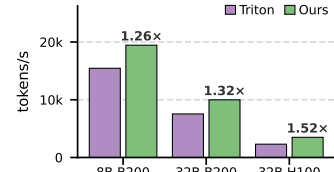


Figure 5: End-to-end throughput on OSWorld trajectories. TriAxialKV delivers 1.26–1.52 $\times$ throughput of Triton’s.

Baselines. We compare against three baselines covering the relevant prior art. SGLang [49] BF16 serves the model with full-precision KV cache and is the lossless reference. SGLang FP4 uses SGLang’s built-in FP4 KV cache backend, representing uniform low-bit floating-point quantization. We also compare with KIVI [26], a seminal asymmetric 2-bit KV cache quantization scheme.

End-to-end serving performance. We measure end-to-end serving throughput in tokens per second on Qwen3-VL-8B and Qwen3-VL-32B, running OSWorld trajectories. The throughput is measured on both a single NVIDIA B200 GPU (180 GB HBM3e) and a single NVIDIA H100 GPU (80 GB HBM3), both hosted in our internal cluster. For each (model, GPU) configuration, we measure the throughput at the largest possible batch size without running out of memory. All experiments are served by SGLang v0.5.10, with our method integrated as modifications.

4.2 Task Accuracy

Tables 2 and 3 report task accuracy on BFCL Memory and OSWorld. Across all experiments, our mixed-precision allocation **tracks the BF16 reference closely**: on BFCL Memory it lands within 1.1% of the BF16 baseline on every model, and on OSWorld within 1.8% on every model. SGLang FP4 is unstable across models. It lands well above BF16 on Falcon3-10B but well below on Qwen3-14B and Qwen3-32B, suggesting that uniform low-bit floating-point quantization interacts unpredictably with model-specific weight and activation distributions. Our mixed allocation avoids both failure modes by spending the available bits where the calibrated sensitivity table indicates they are needed. Per-category accuracy is included in Appendix B.

The contrast between our TriAxialKV Mixed and KIVI on BFCL Memory **isolates the value of per-tag allocation**. KIVI applies uniform 2-bit quantization to all non-residual tokens, including the system prompt and tool schemas that the calibration sweep identifies as the most sensitive tag in the taxonomy. These tokens carry the function signatures and argument specifications that the model *must reproduce verbatim in its tool calls*, and small quantization errors in their KV representations propagate into incorrect argument names or types, which BFCL Memory scores as task failures. Our mixed allocation spends the 4-bit budget on exactly this *inst*-tagged segment while pushing some other segments to 2-bit, recovering most of the accuracy that uniform 2-bit quantization gives up.

Table 4: Axis ablation on BFCL Memory. Removing either axis leads to accuracy degradation.

	Qwen3-14B	Qwen3-32B
No Temporal	22.00	24.00
No Semantic	18.00	20.89
Full	24.22	25.11

Table 5: Memory budget sweep from 2.5 to 2.7 bits on Qwen3-14B (BFCL Memory).

Method	Average bitwidth B		
	2.5	2.6	2.7
TriAxialKV Mixed	16.22	19.56	24.22

4.3 Serving Throughput

To measure the serving throughput, we replayed OSWorld trajectories, which have an average prefill length of 11,000 tokens and an average decode length of 300 tokens. All methods use FlashInfer [43] for prefill; the SGLang BF16 baseline uses SGLang’s original Triton decode kernel, while our method uses our custom fused Triton kernel for mixed-precision dequantization and attention. Figure 5 reports end-to-end throughput at the maximum concurrency each method can sustain. Our method delivers **1.26× the BF16 throughput** on Qwen3-VL-8B (B200), **1.32×** on Qwen3-VL-32B (B200), and **1.52×** on Qwen3-VL-32B (H100). At the same memory budget, our method sustains **3.4-4.0× average concurrent in-flight requests** (Qwen3-VL-32B on H100: 11.78 vs. 3.46; on B200: 118.82 vs. 29.79; Qwen3-VL-8B on B200: 295.76 vs. 84.06), and the larger concurrency better saturates decode compute. The throughput advantage is largest on H100 (1.52×), where lower HBM bandwidth makes the compressed KV cache most valuable, and smaller on B200 (1.26-1.32×), whose abundant memory narrows the headroom compression can recover.

4.4 Ablation Study

Axis ablation. Table 4 ablates the temporal and semantic axes of the taxonomy at the calibrated average bitwidth on BFCL Memory. *No Temporal* treats all tokens as belonging to a single turn, removing temporal recency from the tag space; *No Semantic* treats all tokens as a single semantic role, regardless of whether they are system prompts, user queries, tool calls, or assistant prose. Both axes contribute, but the semantic axis dominates: removing it drops accuracy by 6.22 points on Qwen3-14B and 4.22 points on Qwen3-32B, roughly three times the impact of removing temporal recency. The gap reflects what each axis lets the allocator do. The semantic axis isolates segments like the system prompt and tool schemas, where errors in KV propagate into incorrect argument names, leading to task failures. Collapsing this axis forces a uniform bitwidth across all roles and starves the most fragile segment of bits. The temporal axis adds a smaller but consistent gain by letting the allocator quantize older turns aggressively, since attention concentrates on the current turn.

Memory budget. Table 5 sweeps the average bitwidth B on Qwen3-14B (BFCL Memory) from the calibrated bitwidth 2.7 down to 2.5. Accuracy drops sharply from 24.22 at 2.7 to 19.56 at 2.6 and 16.22 at 2.5, with each tenth-bit reduction losing roughly 5% accuracy. The steep slope near the calibrated point justifies the calibration sweep of Section 3.1 as deliberate operating-point selection rather than a hyperparameter that could be set conservatively without cost.

5 Conclusion

We presented TriAxialKV, a mixed-precision KV-cache quantization framework that exploits the structured heterogeneity of agentic prefills along three orthogonal axes: temporal recency, modality, and semantic role. Our contributions are a chat-template-only tagger that labels prefill tokens in a single pass, a calibration procedure that measures per-tag attention-output distortion, and a bitwidth allocator that allocates INT2/INT4 bitwidths under a memory budget. We integrated these with a paged INT2/INT4 memory pool and fused Triton decode kernels into SGLang. On BFCL Memory and OSWorld, TriAxialKV preserves the task accuracies compared to the BF16 reference. On Qwen3-VL-32B (OSWorld), it fits a 4.5× KV cache and delivers 1.32× the BF16 throughput on B200 and 1.52× on H100. These results show that jointly reasoning about temporal, modal, and semantic structure is both necessary and practical for aggressive KV-cache compression.

References

- [1] Reyna Abhyankar, Qi Qi, and Yiyang Zhang. OSWorld-Human: Benchmarking the efficiency of computer-use agents. In *ICML Workshop on Computer Use Agents*, 2025.
- [2] Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s2: A compositional generalist-specialist framework for computer use agents. In *COLM*, 2025.
- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve. In *OSDI*, 2024.
- [4] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefer, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated LLMs. In *NeurIPS*, 2024.
- [5] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibo Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yanzhi Zhu, and Ke Zhu. Qwen3-v1 technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [6] Debarag Banerjee, Pooja Singh, Arjun Avadhanam, and Saksham Srivastava. Benchmarking llm-powered chatbots: Methods and metrics. *arXiv preprint arXiv:2308.04624*, 2023.
- [7] Sayed Pedram Haeri Boroujeni, Niloufar Mehrabi, Patrick Woods, Gabriel Hillesheim, and Abolfazl Razi. Don’t waste bits! adaptive KV-cache quantization for lightweight on-device llms. *arXiv preprint arXiv:2604.04722*, 2026.
- [8] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, et al. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. In *COLM*, 2025.
- [9] Hyungjoo Chae, Namyoung Kim, Kai Tzu-iunn Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation. In *ICLR*, 2025.
- [10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [11] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *ICLR*, 2024.
- [12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.
- [13] Michael Davies, Neal Crago, Karthikeyan Sankaralingam, and Christos Kozyrakis. Liminal: Exploring the frontiers of llm decode performance. *arXiv preprint arXiv:2507.14397*, 2025.
- [14] Falcon-LLM Team. The falcon 3 family of open models, 2024.
- [15] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S Kevin Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. In *NeurIPS*, 2025.
- [16] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. In *ICLR*, 2024.

- [17] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. In *ACL*, 2024.
- [18] Yefei He, Luoming Zhang, Weijia Wu, Jing Liu, Hong Zhou, and Bohan Zhuang. Zipcache: Accurate and efficient kv cache quantization with salient token identification. In *NeurIPS*, 2024.
- [19] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun S Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. In *NeurIPS*, 2024.
- [20] Jinda Jia, Jisen Li, Zhongzhu Zhou, Jung Hwan Heo, Jue Wang, Tri Dao, Shuaiwen Leon Song, Ben Athiwaratkun, Chenfeng Xu, Tianyi Zhang, and Xiaoxia Wu. Saw-int4: System-aware 4-bit kv-cache quantization for real-world llm serving. *arXiv preprint arXiv:2604.19157*, 2026.
- [21] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *SOSP*, 2023.
- [22] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. In *NeurIPS*, 2024.
- [23] Chengxi Liao and Zeyi Wen. Channel-aware mixed-precision quantization for efficient long-context inference. In *ICLR*, 2026.
- [24] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. Qserve: W4a8kv4 quantization and system co-design for efficient llm serving. In *MLSys*, 2025.
- [25] Tengxuan Liu, Shiyao Li, Jiayi Yang, Tianchen Zhao, Feng Zhou, Xiaohui Song, Guohao Dai, Shengen Yan, Huazhong Yang, and Yu Wang. PM-KVQ: Progressive mixed-precision KV cache quantization for long-cot LLMs. In *ICLR*, 2026.
- [26] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. In *ICML*, 2024.
- [27] Jiayi Nie, Haoran Wu, Yao Lai, Zeyu Cao, Cheng Zhang, Binglei Lou, Erwei Wang, Jianyi Cheng, Timothy M. Jones, Robert Mullins, Rika Antonova, and Yiren Zhao. Kernelcraft: Benchmarking for agentic close-to-metal kernel generation on emerging hardware. *arXiv preprint arXiv:2603.08721*, 2026.
- [28] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *ISCA*, 2024.
- [29] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *ICML*, 2025.
- [30] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *FAST*, 2025.
- [31] Akshat Ramachandran, Marina Neseem, Charbel Sakr, Rangharajan Venkatesan, Brucek Khailany, and Tushar Krishna. Thinkv: Thought-adaptive kv cache compression for efficient reasoning models. In *ICLR*, 2026.
- [32] Stefano Rando, Luca Romani, Alessio Sampieri, Luca Franco, John Yang, Yuta Kyuragi, Fabio Galasso, and Tatsunori Hashimoto. Longcodebench: Evaluating coding LLMs at 1m context windows. In *COLM*, 2025.

- [33] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. In *NeurIPS*, 2024.
- [34] Qian Tao, Wenyuan Yu, and Jingren Zhou. AsymKV: Enabling 1-bit quantization of KV cache with layer-wise asymmetric quantization configurations. In *COLING*, 2025.
- [35] Philippe Tillet and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.
- [36] Dezhan Tu, Danylo Vashchilenko, Yuzhe Lu, and Panpan Xu. VL-cache: Sparsity and modality-aware KV cache compression for vision-language model inference acceleration. In *ICLR*, 2025.
- [37] Zhongwei Wan, Ziang Wu, Che Liu, Jinfa Huang, Zhihong Zhu, Peng Jin, Longyue Wang, and Li Yuan. LOOK-M: Look-once optimization in KV cache for efficient multimodal long-context inference. In *Findings of EMNLP*, 2024.
- [38] Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, et al. Internvl3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. *arXiv preprint arXiv:2508.18265*, 2025.
- [39] Haoran Wu, Can Xiao, Jiayi Nie, Xuan Guo, Binglei Lou, Jeffrey T. H. Wong, Zhiwen Mo, Cheng Zhang, Przemyslaw Forys, Chengyang Ai, Timi Adeniran, Wayne Luk, Hongxiang Fan, Jianyi Cheng, Timothy M. Jones, Rika Antonova, Robert Mullins, and Aaron Zhao. Combating the memory walls: Optimization pathways for long-context agentic llm inference. *arXiv preprint arXiv:2509.09505*, 2026.
- [40] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *ICLR*, 2024.
- [41] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *NeurIPS*, 2024.
- [42] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [43] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. Flashinfer: Efficient and customizable attention engine for llm inference serving. *MLSys*, 2025.
- [44] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *OSDI*, 2022.
- [45] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling. In *MLSys*, 2026.
- [46] Zhongshen Zeng, Pengguang Chen, Shu Liu, Haiyun Jiang, and Jiaya Jia. MR-GSM8K: A meta-reasoning benchmark for large language model evaluation. In *ICLR*, 2025.
- [47] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023.
- [48] Yi Zhao, Yajuan Peng, Nguyen Cam-Tu, Zuchao Li, Wang Xiaoliang, Xiaoming Fu, et al. Smallkv: Small model assisted compensation of kv cache compression for efficient llm inference. In *NeurIPS*, 2025.

- [49] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. In *NeurIPS*, 2024.
- [50] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *OSDI*, 2024.

A Greedy Bitwidth Allocation

Algorithm 1 Semantic-aware bit allocation

Require: Tag set $\mathcal{S}$; counts $\{N_s\}$; distortions $\{D_s(2), D_s(4)\}$; budget B .

Ensure: Allocation $\mathbf{b}^* \in \{2, 4\}^{|\mathcal{S}|}$.

- 1: Compute $\rho_s \leftarrow (D_s(2) - D_s(4)) / (2N_s)$ for all $s \in \mathcal{S}$.
 - 2: Sort $\mathcal{S}$ by ρ_s descending; set $b_s \leftarrow 2$ and $R \leftarrow (B - 2) \sum_s N_s$.
 - 3: **for** s in sorted order **do**
 - 4: **if** $2N_s \leq R$ **then**
 - 5: $b_s \leftarrow 4$; $R \leftarrow R - 2N_s$.
 - 6: **end if**
 - 7: **end for**
 - 8: Enumerate $\mathbf{b} \in \{2, 4\}^{|\mathcal{S}|}$ satisfying the budget; return $\arg \min \sum_s D_s(b_s)$.
-

B Per-Category Accuracy on BFCL and OSWorld

Table 6: Per-category accuracy (%) on BFCL Memory across the KV, Vector, and RecSum sub-tasks. Parenthesized values give the fraction of prefill tokens routed to INT2 under the calibrated allocation.

Method	Overall	KV	Vector	RecSum
<i>Falcon3-10B-Instruct</i>				
SGLang BF16	24.00	10.00	14.67	47.33
SGLang FP4	28.22	14.00	16.67	54.00
KIVI	22.89	15.33	16.67	36.67
TriAxialKV INT4	22.00	8.67	12.67	44.67
TriAxialKV Mixed (75% INT2)	24.00	10.67	16.67	44.67
<i>Qwen3-14B</i>				
SGLang BF16	25.11	13.33	19.33	42.67
SGLang FP4	18.00	8.00	8.67	37.33
KIVI	20.67	4.00	15.33	42.67
TriAxialKV INT4	24.89	10.67	20.00	44.00
TriAxialKV Mixed (65% INT2)	24.22	12.00	11.33	49.33
<i>Qwen3-32B</i>				
SGLang BF16	25.78	16.00	17.33	44.00
SGLang FP4	20.22	14.00	15.33	31.33
KIVI	21.56	16.67	16.00	32.00
TriAxialKV INT4	24.67	16.67	16.00	41.33
TriAxialKV Mixed (75% INT2)	25.11	12.00	22.67	40.67
<i>Qwen3-235B-A22B-Instruct-2507</i>				
SGLang BF16	23.78	10.67	16.67	44.00
SGLang FP4	23.56	12.67	12.67	45.33
KIVI	23.33	9.33	16.67	44.00
TriAxialKV INT4	23.11	12.00	12.00	45.33
TriAxialKV Mixed (75% INT2)	22.67	12.67	18.00	37.33

Table 7: Per-category accuracy (%) on OSWorld across the OS, Office, Daily, Professional, and Workflow application categories. TriAxialKV Mixed matches BF16 accuracy on most categories.

Method	Overall	OS	Office	Daily	Professional	Workflow
<i>Qwen3-VL-8B-Thinking</i>						
SGLang BF16	34.02	47.83	27.93	50.00	48.94	16.28
SGLang FP4	29.52	50.00	25.77	43.08	31.82	13.06
KIVI	34.61	43.48	30.30	44.27	51.06	20.86
TriAxialKV INT4	33.92	39.13	31.53	47.95	48.94	15.91
TriAxialKV Mixed	35.78	43.48	32.97	52.72	40.43	21.00
<i>Qwen3-VL-32B-Thinking</i>						
SGLang BF16	39.20	50.00	33.86	53.54	48.98	25.83
SGLang FP4	40.54	43.48	39.22	56.43	48.94	23.60
KIVI	41.49	56.52	39.21	47.10	57.45	27.21
TriAxialKV INT4	41.43	52.17	44.59	49.51	48.94	23.94
TriAxialKV Mixed	40.59	43.48	37.41	52.59	53.19	27.35
<i>InternVL3.5-38B</i>						
SGLang BF16	12.22	21.74	6.30	21.51	21.28	4.55
SGLang FP4	12.13	0.00	7.97	23.59	17.02	2.04
KIVI	12.26	21.74	5.48	28.11	10.87	5.68
TriAxialKV INT4	13.11	23.81	5.40	24.22	21.28	6.10
TriAxialKV Mixed	12.77	26.09	6.27	24.20	19.15	4.55

C Limitations

Our method has a few limitations:

- The per-tag sensitivity table is calibrated per workload and per model, requiring a one-time offline pass over a small calibration set. While inexpensive, this means the method is not zero-shot transferable to entirely new workloads.
- Our tagger relies on chat-template structure such as role markers, message delimiters, and brackets, and requires adaptation for models or prompting styles that lack standardized markers.
- Our allocator currently selects between INT2 and INT4 only. Supporting finer-grained bitwidths could yield further Pareto improvements, but would complicate kernel and memory-pool design.

D Broader Impacts

TriAxialKV is a KV-cache compression technique that reduces the memory and energy cost of serving large language models. We see two primary **positive** societal impacts. First, lower memory consumption per request enables larger batch sizes and higher throughput on the same hardware, reducing the per-token energy cost of inference and the corresponding environmental footprint of large-scale LLM deployment. Second, by allowing capable models to fit within the memory budget of less-expensive GPUs, our method lowers the hardware barrier to deploying agentic systems, broadening access for researchers and practitioners with limited compute resources. As far as we are aware, our work does not introduce new negative societal impacts beyond those already inherent to the underlying models being served. TriAxialKV does not modify model weights, training data, or output behavior; it is a serving-system optimization that preserves task accuracy relative to the BF16 reference. Any risks associated with the deployment of LLMs or agentic systems are inherited from the base models, and existing safeguards for those models continue to apply.

E Licenses for Existing Assets

We list below the existing assets used in this work, along with their licenses. All assets are used in accordance with their respective license terms.

Models:

- Qwen3 (14B, 32B, 235B-A22B-Instruct-2507) [42]: Apache License 2.0.
- Qwen3-VL (8B-Thinking, 32B-Thinking) [5]: Apache License 2.0.
- Falcon3-10B-Instruct [14]: Falcon LLM License (TII Falcon License 2.0).
- InternVL3.5-38B [38]: Apache License 2.0.

Benchmarks and datasets:

- Berkeley Function Calling Leaderboard (BFCL) [29]: Apache License 2.0.
- OSWorld [41]: Apache License 2.0.

Serving frameworks and libraries:

- SGLang [49] (v0.5.10): Apache License 2.0. Our implementation is built as a fork of SGLang.
- FlashInfer [43]: Apache License 2.0. Used as the FP16 prefill attention backend.
- Triton [35]: MIT License. Used to implement our fused mixed-precision decode kernels.
- KIVI [26]: MIT License. Used as a baseline for comparison.